

Module 6

Q. Desirable Features of Distributed File System

=>

1. A distributed file system allows files to be stored and accessed across multiple machines in a network.
2. The main goal of a distributed file system is to provide users with easy and reliable file access.
3. A good distributed file system should have the following desirable features.

Desirable Features of Distributed File System

1. Transparency

1. The distributed file system should hide the fact that files are stored on different machines.
2. Users should access files in the same way as local files.
3. The user should not need to know the physical location of the file.

2. Scalability

1. The distributed file system should support growth in the number of users, files, and servers.
2. Performance should not reduce significantly when the system expands.
3. New machines and storage devices should be added easily.

3. Reliability

1. The distributed file system should continue working even if some servers fail.
2. Data should not be lost due to machine crashes or network failures.
3. Backup and replication techniques should be used to improve reliability.

4. Availability

1. Files should remain accessible even during failures.
2. Replicated copies of files should be used to ensure continuous access.
3. Users should be able to access files whenever needed.

5. Performance

1. The distributed file system should provide fast file access.
2. Caching and replication should be used to reduce access time.
3. File operations should be efficient even when many users access the system.

6. Security

1. The system should protect files from unauthorized access.
2. Authentication and access control should be used.
3. Data encryption should be applied during file transfer.

7. Consistency

1. All users should see the correct and updated version of a file.
2. Multiple copies of files should remain synchronized.
3. File consistency should be maintained during concurrent access.

8. Fault Tolerance

1. The system should recover automatically from failures.
2. File replication and redundancy should be used.
3. The system should continue operating even if one server crashes.

9. Heterogeneity

1. The distributed file system should support different operating systems and hardware platforms.
2. Users on Windows, Linux, and macOS should access files easily.
3. The system should support communication between different machine types.

10. File Sharing

1. The distributed file system should allow multiple users to share files.
2. Shared access should be controlled properly to avoid conflicts.
3. File locking mechanisms should be provided.

Modification Propagation in File Caching Schemes

1. File caching is used to store copies of files locally for faster access.
2. When a cached file is modified, the changes must be propagated to other copies to maintain consistency.
3. Modification propagation means updating all cached copies after a file change.

1. Write-Through Cache

1. In write-through caching, every modification is immediately written to the main server.
2. This method provides strong consistency.
3. However, write-through caching may reduce performance because every write operation contacts the server.
4. An example is updating a file and immediately saving changes to the server.

2. Delayed Write or Write-Back Cache

1. In write-back caching, modifications are first stored in the local cache.
2. The updated file is written back to the server later.
3. This method improves performance because fewer server accesses are required.
4. However, data may be lost if the client crashes before writing changes back.

3. Invalidate Propagation

1. In invalidate propagation, when one client updates a file, other cached copies are marked invalid.
2. Other clients must fetch the updated file from the server before using it again.
3. This method reduces unnecessary data transfer.
4. An example is a file editing system where other users must reload the file after an update.

4. Update Propagation

1. In update propagation, the updated file contents are sent to all cached copies immediately.
2. All clients receive the latest version of the file.
3. This method provides better consistency.
4. However, it increases network traffic because updates are sent to many clients.

Q. Explain Synchronization in DFS with its Challenges

=>

Synchronization in Distributed File System (DFS)

1. Synchronization in a Distributed File System means coordinating access to shared files among multiple users or processes.
2. Synchronization ensures that multiple users do not make conflicting changes to the same file at the same time.
3. Synchronization helps maintain consistency and correctness of file data.
4. In DFS, files may be stored on different servers and accessed by many clients simultaneously.
5. Proper synchronization prevents problems such as lost updates, incorrect data, and file corruption.
6. Synchronization is usually achieved using locking, timestamps, version control, or transaction mechanisms.

Need for Synchronization in DFS

1. Many users may try to access the same file at the same time.
2. Some users may want to read a file while others want to modify it.
3. Without synchronization, one user's update may overwrite another user's changes.
4. Synchronization ensures that all users see a correct and consistent version of the file.
5. Synchronization is especially important in collaborative systems and shared databases.

Methods of Synchronization in DFS

1. File Locking

1. File locking is a common synchronization method in DFS.
2. A process locks a file before modifying it.
3. Other processes cannot modify the locked file until the lock is released.
4. There are two types of file locks.

a. Shared Lock

1. Shared lock allows multiple users to read a file at the same time.
2. No user can modify the file while it is under shared lock.
3. Shared lock is also called read lock.

b. Exclusive Lock

1. Exclusive lock allows only one user to read and write the file.
2. Other users cannot access the file until the lock is released.
3. Exclusive lock is also called write lock.

2. Timestamp-Based Synchronization

1. Timestamps are used to record the order of file modifications.
2. The system compares timestamps to decide which version of the file is latest.
3. This method helps avoid conflicts during concurrent updates.

3. Version Control

1. Version control stores multiple versions of the same file.
2. If conflicts occur, users can compare different versions and merge changes.
3. This method is useful in collaborative software development systems.

4. Transaction-Based Synchronization

1. In transaction-based synchronization, a group of file operations is treated as a single transaction.
2. Either all operations are completed successfully or none are applied.
3. This ensures consistency of data.

Challenges in Synchronization in DFS

1. Concurrent Access Problem

1. Multiple users may try to modify the same file at the same time.
2. This may lead to inconsistent or corrupted data.
3. Proper locking mechanisms are needed to avoid conflicts.

2. Deadlock Problem

1. Deadlock occurs when two or more processes wait for each other to release file locks.
2. Deadlocks can stop file operations completely.
3. Deadlock detection and prevention techniques are needed.

3. Network Delay

1. In distributed systems, communication between clients and servers may be delayed.
2. Delays may cause outdated file versions to be accessed.
3. Network latency can reduce synchronization efficiency.

4. Cache Consistency Problem

1. Clients may store local cached copies of files.
2. A cached copy may become outdated if another client updates the file.
3. Modification propagation mechanisms are needed to maintain cache consistency.

5. Fault Tolerance Problem

1. System crashes or network failures may occur during synchronization.
2. Locks may remain active even if the process holding them crashes.
3. Recovery mechanisms are required to handle failures.

6. Scalability Problem

1. As the number of users and files increases, synchronization becomes more difficult.
2. Large systems may experience heavy communication overhead.
3. Efficient synchronization methods are needed for scalability.

7. Heterogeneity Problem

1. Different machines may use different operating systems and file formats.
2. Synchronization becomes difficult when different platforms interact.
3. The DFS must support interoperability between systems.

Q. Explain File Caching Schemes

=>

File Caching

1. File caching is a technique used in distributed file systems to improve file access speed.
2. In file caching, a copy of the file is stored temporarily in local memory or disk.
3. Cached files can be accessed quickly without contacting the server every time.
4. File caching reduces network traffic and improves system performance.
5. File caching is useful when the same file is accessed repeatedly.

Types of File Caching Schemes

1. Server-Side Caching

1. In server-side caching, frequently accessed files are stored in the server's main memory.
2. The server can provide requested files quickly without reading from disk every time.
3. This method improves response time for all clients.
4. Server-side caching reduces disk access overhead.
5. An example is a file server keeping recently accessed files in RAM.

2. Client-Side Caching

1. In client-side caching, copies of files are stored in the client machine.
2. Clients can access cached files locally without contacting the server.
3. This method reduces network communication and improves access speed.
4. Client-side caching is commonly used in distributed file systems.
5. An example is a user opening the same document multiple times without downloading it again.

Based on Cache Storage Location

1. Memory Cache

1. Memory cache stores file data in the main memory (RAM).
2. Accessing data from RAM is very fast.
3. Memory cache improves file read speed significantly.
4. However, cached data is lost if the system shuts down.
5. An example is storing recently used files in RAM.

2. Disk Cache

1. Disk cache stores file data on the local disk of the client or server.
2. Disk cache is slower than memory cache but can store larger amounts of data.
3. Disk cache is not lost after system restart.
4. An example is browser cache stored on a hard drive.

Based on Data Unit Cached

1. File-Level Caching

1. In file-level caching, the entire file is stored in the cache.
2. This method is useful when the complete file is frequently accessed.
3. File-level caching may require more storage space.
4. An example is caching an entire video file locally.

2. Block-Level Caching

1. In block-level caching, only some blocks or parts of the file are cached.
2. This method saves storage space because only required blocks are stored.
3. Block-level caching is useful for large files.
4. An example is caching only the required pages of a database file.

Based on Update Policy

1. Write-Through Cache

1. In write-through caching, every modification is immediately written to both cache and server.
2. This method provides better consistency.
3. However, it may reduce performance because every write contacts the server.
4. An example is immediately saving changes to a shared document server.

2. Write-Back Cache

1. In write-back caching, modifications are first stored in cache and written to the server later.
2. This method improves performance because fewer server accesses are needed.
3. However, data may be lost if the client crashes before updating the server.
4. An example is saving changes locally and syncing them later.

Advantages of File Caching

1. File caching improves file access speed.
2. File caching reduces network traffic.
3. File caching decreases server load.
4. File caching improves overall system performance.
5. File caching allows repeated access to files without repeated downloads.

Disadvantages of File Caching

1. Cached files may become outdated if another client modifies the original file.
2. Cache consistency must be maintained carefully.
3. File caching may require extra memory or disk space.
4. Synchronization between cached copies can be difficult.